Nice, this is a perfect spec to turn into a Replit build brief 👌

Below is something you can **copy–paste directly into Replit's AI / dev agent** so it knows exactly what to code, end-to-end. I'll write it as instructions to the coder.

🔧 FULL REPLIT BUILD INSTRUCTIONS

Project: Fasil Kenema Fan Club Platform
Stack (fixed, please follow):

- **Backend:** Node.js + Express
- **ORM:** Prisma
- **Database:** PostgreSQL (use Replit DB/Postgres or external managed Postgres)
- **Frontend (Public site + Admin panel):** React + Vite (single project with routes)
- **UI Library (Admin):** Ant Design or MUI
- **Auth:** JWT-based auth for admins & members
- **Payment:** Chapa Payment Gateway (sandbox + production)
- **SMS:** Geez SMS HTTP API
- **Language:** TypeScript (backend + frontend)

---

1. Project Setup

1. Create a **Node.js + React (Vite)** monorepo-style project in Replit:
    - `/backend` → Express + Prisma
    - `/frontend` → React + Vite
2. Add scripts in root `package.json`:

- `"dev:backend": "cd backend && nodemon src/index.ts"`
- `"dev:frontend": "cd frontend && npm run dev"`
- `"dev": "concurrently \"npm run dev:backend\" \"npm run dev:frontend\""`

3. Configure CORS so frontend (e.g. `http://localhost:5173`) can consume backend.

---

2. Environment Variables

Create a `.env` file in `/backend` with:

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:PORT/DB_NAME
JWT_SECRET=your-strong-secret
CHAPA_SECRET_KEY=chapa-secret-key
CHAPA_PUBLIC_KEY=chapa-public-key
CHAPA_BASE_URL=https://api.chapa.co
GEEZ_SMS_API_KEY=your-geez-sms-key
GEEZ_SMS_SENDER_ID=FASILFC
FRONTEND_BASE_URL=http://localhost:5173
APP_BASE_URL=http://localhost:3000
DEFAULT_LANGUAGE=en
```

Make the backend read these with `dotenv`.

---

3. Database Schema (Prisma)

Create a Prisma schema in `/backend/prisma/schema.prisma` with **at least** the following models:

3.1 Core Users & Roles

```
model AdminUser {
  id        Int      @id @default(autoincrement())
  name      String
  email     String   @unique
```

```
  password  String
  role      AdminRole
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

enum AdminRole {
  SUPER_ADMIN
  CONTENT_ADMIN
  MEMBERSHIP_ADMIN
  FINANCE_ADMIN
}

model Member {
  id             Int       @id @default(autoincrement())
  firstName      String
  lastName       String
  phone          String
  email          String?
  city           String?
  region         String?
  preferredLang  String    // 'en', 'am', 'ti', 'om', 'so'
  tier           MembershipTier @default(REGULAR)
  status         MembershipStatus @default(PENDING)
  membershipId   String    @unique
  expiryDate     DateTime?
  digitalCardUrl  String?
  nfcCardUid     String?   @unique
  createdAt      DateTime  @default(now())
  updatedAt      DateTime  @updatedAt
  payments       MembershipPayment[]
  donations      Donation[]
}

enum MembershipTier {
  REGULAR
  PREMIUM
  VIP
}

enum MembershipStatus {
  PENDING
  ACTIVE
  EXPIRED
```

```
  BLOCKED
}
```

3.2 Payments & Donations

```
model MembershipPayment {
  id             Int      @id @default(autoincrement())
  memberId       Int
  member         Member   @relation(fields: [memberId], references: [id])
  chapaTxRef     String   @unique
  amount         Float
  currency       String   // 'ETB' or 'USD'
  status         PaymentStatus
  paidAt         DateTime?
  createdAt      DateTime @default(now())
}

model Donation {
  id             Int      @id @default(autoincrement())
  donorName      String?
  phone          String?
  email          String?
  chapaTxRef     String   @unique
  amount         Float
  currency       String   // 'ETB' or 'USD'
  isAnonymous    Boolean  @default(false)
  status         PaymentStatus
  purpose        String?
  createdAt      DateTime @default(now())
  memberId       Int?
  member         Member?  @relation(fields: [memberId], references: [id])
}

enum PaymentStatus {
  PENDING
  SUCCESS
  FAILED
}
```

3.3 Content & Translations

```
model Page {
  id         Int      @id @default(autoincrement())
```

```
  slug        String   @unique   // 'home', 'about', 'contact'
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  sections    PageSection[]
}

model PageSection {
  id        Int      @id @default(autoincrement())
  pageId    Int
  page      Page     @relation(fields: [pageId], references: [id])
  key       String   // 'hero', 'highlights', 'footer', etc.
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  contents  PageSectionContent[]
}

model PageSectionContent {
  id          Int      @id @default(autoincrement())
  sectionId   Int
  section     PageSection @relation(fields: [sectionId], references: [id])
  language    String   // 'en', 'am', 'ti', 'om', 'so'
  title       String?
  subtitle    String?
  body        String?
  imageUrl    String?
  buttonLabel String?
  buttonLink  String?
}
```

3.4 Blog / News

```
model Post {
  id          Int      @id @default(autoincrement())
  slug        String   @unique
  category    String
  coverImage  String?
  authorName  String
  publishedAt DateTime?
  isPublished Boolean   @default(false)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  translations PostTranslation[]
}
```

```
model PostTranslation {
  id        Int      @id @default(autoincrement())
  postId    Int
  post      Post     @relation(fields: [postId], references: [id])
  language  String   // 'en', 'am', 'ti', 'om', 'so'
  title     String
  content   String   // HTML or markdown
}
```

3.5 SMS & Templates

```
model SmsTemplate {
  id        Int      @id @default(autoincrement())
  key       String   @unique // 'WELCOME', 'RENEWAL_REMINDER',
'DONATION_THANK_YOU'
  language  String   // 'en', 'am', 'ti', 'om', 'so'
  body      String   // supports placeholders like {{name}}, {{amount}}, {{membershipId}}
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model SmsLog {
  id        Int      @id @default(autoincrement())
  phone     String
  message   String
  status    String   // 'SENT', 'FAILED'
  providerId String?
  createdAt DateTime @default(now())
}
```

3.6 Global Settings

```
model Setting {
  id        Int      @id @default(autoincrement())
  key       String   @unique
  value     String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Use `Setting` for:

- `SITE_PRIMARY_COLOR`
- `SITE_LOGO_URL`
- `DEFAULT_LANGUAGE`
- `MEMBERSHIP_FEE_REGULAR_ETB`
- `MEMBERSHIP_FEE_REGULAR_USD`
- etc.

Run `npx prisma migrate dev` after defining schema.

---

4. Backend API Design

Create an Express app with these route groups (all JSON):

4.1 Auth

- `POST /api/admin/auth/login`
    - body: `{ email, password }`
    - returns: `{ token, user }`
- Protect all `/api/admin/**` routes with JWT middleware + role checking.

4.2 Public Content

- `GET /api/public/home?lang=en`
    - Returns all sections for page `home` with the given language.
- `GET /api/public/about?lang=en`

- `GET /api/public/contact?lang=en`
- `GET /api/public/posts?lang=en&category=...`
- `GET /api/public/posts/:slug?lang=en`

4.3 Membership

- `POST /api/public/membership/register`
    - body: `{ firstName, lastName, phone, email, city, region, preferredLang, tier, currency }`
    - Generates `membershipId`, saves member with `PENDING`, creates `MembershipPayment` with `PENDING`.
    - Calls Chapa initialize endpoint and returns checkout URL to frontend.
- `GET /api/public/membership/chapa/callback`
    - Handle Chapa redirect / webhook.
    - Verify with Chapa verify endpoint.
    - On success:
        - Update `MembershipPayment` to `SUCCESS`
        - Update `Member.status = ACTIVE`, set `expiryDate`
        - Generate digital card URL (simple page with QR code) and store in `Member.digitalCardUrl`
        - Trigger welcome SMS using Geez SMS + email.
- `GET /api/member/:membershipId/card`
    - Returns data for digital card (member info + QR encoded data).

4.4 Donation

- `POST /api/public/donations/initiate`
    - body: `{ donorName, phone, email, amount, currency, purpose, isAnonymous }`
    - Create `Donation` with `PENDING`, initialize payment via Chapa, return checkout URL.
- `GET /api/public/donations/chapa/callback`
    - Verify payment, update `Donation` to `SUCCESS` or `FAILED`
    - Send “thank you” SMS if phone provided.

4.5 Admin – Content Management

All require JWT and relevant role.

- `GET /api/admin/pages` – list all pages
- `GET /api/admin/pages/:slug/sections` – get sections + localized contents
- `PUT /api/admin/pages/:slug/sections`
    - body: array of `{ key, language, title, subtitle, body, imageUrl, buttonLabel, buttonLink }`
    - This is where **every text/image on Home/About/Contact** can be edited.
- `POST /api/admin/posts` – create new post with translations
- `PUT /api/admin/posts/:id`
- `DELETE /api/admin/posts/:id`
- `GET /api/admin/posts`

4.6 Admin – Membership & Donations

- `GET /api/admin/members?status=ACTIVE&lang=am&city=...`
- `GET /api/admin/members/:id`
- `PUT /api/admin/members/:id` (update tier, status, etc.)
- `GET /api/admin/payments/memberships?from=...&to=...`
- `GET /api/admin/payments/donations?from=...&to=...`
- `GET /api/admin/export/members` (CSV)
- `GET /api/admin/export/donations` (CSV)

4.7 Admin – SMS

- `GET /api/admin/sms/templates`
- `POST /api/admin/sms/templates`
- `PUT /api/admin/sms/templates/:id`
- `POST /api/admin/sms/send/broadcast`
    - body: `{ segmentType, filters, templateKey }`
- `GET /api/admin/sms/logs`

Implement a helper `sendSms(phone, message)` to call Geez SMS API and log to `SmsLog`.

4.8 Admin – Settings

- `GET /api/admin/settings`
- `PUT /api/admin/settings`

    - Allows changing:
        - membership fees
        - default language
        - site brand colors
        - logo URLs
        - Chapa keys (optional)
        - SMS sender ID

---

5. Frontend – Public Website (React + Vite)

Create routes:

- `/` → Home
- `/about` → About Us
- `/contact` → Contact Us
- `/blog` → Blog & News list
- `/blog/:slug` → Blog details
- `/membership` → Membership registration page
- `/donate` → Donation page
- `/card/:membershipId` → Digital membership card display

5.1 Multi-Language

- Use `i18next` or a simple context for language.

- Language switcher in the header: EN, አማ, ትግ, Afaan Oromoo, Soomaali.
- On route load, call `GET /api/public/{page}?lang=currentLang` to get dynamic content.

5.2 Home Page

- Fetch `/api/public/home?lang=xx`.
- Render hero section:
    - image/background
    - title, subtitle from `PageSectionContent`
    - "Become a Member" and "Donate Now" buttons link to `/membership` and `/donate`.
- Render highlight cards:
    - Latest posts via `/api/public/posts?lang=xx&limit=3`
    - Upcoming events section placeholder (configurable in CMS).
- Embedded social media:
    - Use iframes or links for Facebook/X/YouTube.

5.3 About Page

- Fetch `/api/public/about?lang=xx`.
- Sections:
    - History
    - Mission & Vision
    - Leadership cards

    - Milestones / gallery

All text & images must be fetched via CMS (admin can change it).

5.4 Contact Page

- Fetch `/api/public/contact?lang=xx`.
- Show:
    - Address, phone, email
    - Embedded map (Google Map iframe or static image URL from admin)
- Contact form:
    - `POST /api/public/contact` (optional logging table)
    - Show success/failure message.

5.5 Blog & News

- `/blog`:
    - Fetch `GET /api/public/posts?lang=xx`
    - List with search + category filter.
- `/blog/:slug`:
    - Fetch `GET /api/public/posts/:slug?lang=xx`.

---

6. Frontend – Membership & Donation Flows

6.1 Membership Page `/membership`

- Form with:

    - Name, surname
    - Phone (Ethiopian format validation + optional intl)
    - Email
    - City, region
    - Preferred language
    - Tier dropdown (Regular, Premium, VIP)
    - Currency (ETB / USD)
- On submit:
    - Call `POST /api/public/membership/register`
    - Receive Chapa checkout URL
    - Redirect user there.
- After Chapa payment:
    - Chapa returns to `FRONTEND_BASE_URL + "/membership/thanks?tx_ref=..."`
    - Call backend to verify if not done via webhook, display success.

6.2 Digital Card `/card/:membershipId`

- Fetch `GET /api/member/:membershipId/card`
- Show:
    - Name, Tier, Membership ID, Expiry
    - QR code generated on frontend (e.g. `qrcode.react`) pointing to `member verification endpoint` or simply encoding membership ID.

6.3 Donation Page `/donate`

- Form:
    - Name (optional)
    - Phone
    - Email
    - Amount
    - Currency
    - Purpose
    - Checkbox “Donate anonymously”
- On submit:
    - `POST /api/public/donations/initiate`
    - Redirect to Chapa.

---

7. Frontend – Admin Dashboard

Create separate React routes under `/admin`:

- `/admin/login`
- `/admin/dashboard`
- `/admin/pages/home`
- `/admin/pages/about`
- `/admin/pages/contact`

- `/admin/posts`
- `/admin/posts/new`
- `/admin/posts/:id/edit`
- `/admin/members`
- `/admin/members/:id`
- `/admin/donations`
- `/admin/payments`
- `/admin/sms/templates`
- `/admin/sms/broadcast`
- `/admin/settings`
- `/admin/admin-users` (only for SUPER_ADMIN)

7.1 Admin – Page Customization (VERY IMPORTANT)

For **Home**, **About**, **Contact** pages:

- Build a UI where admin can:
    - Switch language (EN, AM, etc.)
    - Edit:
        - Hero title, subtitle, background image URL
        - Button texts and links
        - Sections (text, list items, icons, etc.)
    - Add/remove sections (or at least update existing pre-defined ones).

- Changes are saved via `PUT /api/admin/pages/:slug/sections`.

Goal:
Every visible text and image on the public pages must come from DB, not hardcoded.

---

8. Chapa Integration (Backend)

Implement a helper module:

```
async function initializeChapaPayment(params: {
  amount: number;
  currency: string;
  tx_ref: string;
  return_url: string;
  callback_url: string;
  first_name?: string;
  last_name?: string;
  email?: string;
}) { ... }

async function verifyChapaPayment(tx_ref: string) { ... }
```

- Use `CHAPA_SECRET_KEY` in `Authorization: Bearer` header.
- Store `tx_ref` in both `MembershipPayment` and `Donation`.

---

9. Geez SMS Integration

Implement:

```
async function sendSms(phone: string, message: string): Promise<void> {
  // Call Geez SMS HTTP API with API key + sender ID
  // On success/failure, save to SmsLog
}
```

Create helper to render templates:

```
async function sendTemplateSms(templateKey: string, language: string, phone: string, data: Record<string, string | number>) {
  // 1. Load SmsTemplate by key + language.
  // 2. Replace {{placeholders}} with data.
  // 3. Call sendSms.
}
```

Use this in:

- Membership success flow → `WELCOME`
- Membership expiry reminder cron (optional) → `RENEWAL_REMINDER`
- Donation success → `DONATION_THANK_YOU`

---

10. Cron Jobs (Optional but Recommended)

Use a simple Node cron (e.g. `node-cron`) for:

- Daily check of memberships nearing expiry (30/15/7 days) → send reminder SMS.
- Daily check for expired memberships to set status to `EXPIRED`.

---

11. NFC Card Handling

Backend:

- In member admin view, allow:
    - Field `nfcCardUid` to be set/updated manually.
- Optionally, add endpoint:
    - `POST /api/admin/members/:id/assign-nfc` with `{ nfcCardUid }`.

- This mapping will be used later by external readers/apps.

---

12. Security & Production Readiness

- Hash all passwords with bcrypt.
- Validate inputs with a validation library (e.g. `zod` or `joi`).
- Handle error responses consistently.
- Use HTTPS in production (Replit / custom domain).
- Configure CORS correctly.

---

13. Final Deliverables

The final Replit project should include:

1. Running backend (Express + Prisma) with migrations in place.
2. Frontend app with:
    - Public pages fully reading content from API/CMS.
    - Multi-language switching functional.
    - Membership & donation flows working with Chapa sandbox.
3. Admin dashboard with:
    - Login for admins.
    - Page content editing for home/about/contact.
    - Blog/news management.

    - Membership & donations views + exports.
    - SMS template and broadcast screens.
    - Settings screen for brand & config.
4. README explaining:
    - How to run backend & frontend.
    - How to set up `.env`.
    - How to run migrations.
    - How to seed initial admin user and default page sections/templates.

If you want, next step I can:

- Generate **sample seed data** (SQL/Prisma seed) for home/about/contact contents in 3 languages, and
- Draft **exact API request/response JSON examples** for each endpoint so your Replit AI writes code exactly as needed.

14. Membership Card Design Requirements (MUST MATCH EXISTING TEMPLATE)

Use these two images as the **official front and back templates** for all membership cards:

- Front template image:
  `/mnt/data/77b37a6f-30ee-41e8-bcbe-b25394254662.jpg`
- Back template image:
  `/mnt/data/baa595fe-652c-445d-b6d2-b8137b119a42.jpg`

Treat them as the **background design**. All member data, QR, and photo are overlaid on top.

14.1 General

- Final card size: standard ID card size **86mm x 54mm** (or equivalent ratio in px).
- Use **exact same colors, fonts style and layout** as in the images (red/white Fasil design).
- Keep all **static Amharic + English texts** as in the templates (do not change wording):
    - “Fasil Kenema Sc Fans Association”
    - “Membership ID Card”
    - Amharic headings at top and bottom, etc.
- Club logos at left/right must stay in the same positions.

14.2 Dynamic Fields – FRONT SIDE

Overlay the following **dynamic text fields** in the existing Amharic/English positions (do not move boxes, just replace sample text):

- `Name` (Amharic + English line)
- `Sex`
- `Address`
- `National ID / የመታ.ቁ` (or membership number if required)
- Membership Year (e.g. `2018` → must be dynamic `year`)
- Membership Fee at bottom (e.g. `36,000 ብር` → configurable from settings)

Also overlay:

- Member **photo placeholder** in the white box on the right.
- The big portrait of the historical figure and other graphics remain from the original background template (part of the background image).

Implementation hint (front):

Import the background image in a React component:

```
 const MembershipCardFront: React.FC<{ member: MemberDto; settings:
CardSettings }> = ({ member, settings }) => {
  return (
    <div className="card-wrapper">
      <img src={frontTemplateUrl} className="card-bg" />
      {/* Absolute-positioned fields */}
      <div className="field name">{member.fullName}</div>
      <div className="field sex">{member.sex}</div>
      <div className="field address">{member.address}</div>
      <div className="field idNo">{member.nationalId ||
member.membershipId}</div>
      <div className="field year">{settings.year}</div>
      <div className="field fee">{settings.feeDisplayText}</div>
      <img src={member.photoUrl} className="photo-slot" />
    </div>
  );
};
```

-
- Use **absolute positioning in CSS** (top/left in px) to match the exact positions in the sample card.

14.3 Dynamic Fields – BACK SIDE

The back side should also be generated from the provided back template image and include:

Static text (as on template):

- Header Amharic line
- Line: “We Certify That The Bearer Of This ID Card is Fasil kenema SC Fans Leader”
- Red bar: “This ID Card is Valid Only For Clubs Supporting Purpose”
- Address, phone, fax, PO box text.
- Email & Facebook text (also configurable from settings if needed).

Dynamic overlays:

- `Address` (city, e.g. Gondar) can be treated as a **setting** editable from admin.
- `Approved by` signature area (keep fixed position, allow uploading signature image from admin).
- **QR code** in the white box on the right:
    - Generate QR from `membershipId` or a verification URL like:
        - `https://<domain>/verify?mid=<membershipId>`
    - The QR must fill the box similar to the sample.

Implementation:

```
const MembershipCardBack: React.FC<{ member: MemberDto; settings:
CardSettings }> = ({ member, settings }) => {
  return (
    <div className="card-wrapper">
      <img src={backTemplateUrl} className="card-bg" />
      {/* Optional dynamic address override */}
      <div className="field address">{settings.addressText}</div>
      {/* QR code */}
      <div className="qr-wrapper">
        <QRCode value={settings.qrBaseUrl + member.membershipId} />
      </div>
      {/* signature image if provided */}
      {settings.approverSignatureUrl && (
        <img src={settings.approverSignatureUrl}
className="signature-img" />
      )}
    </div>
  );
};
```

14.4 Digital Card View

- Member portal should have a **“View Card”** screen showing:

    - Card front by default.
    - Toggle / flip button to see card back.
- Use the same components (`MembershipCardFront` and `MembershipCardBack`) rendered at screen resolution.
- Also provide a **"Download as PDF"** or "Print Card" button:
    - Use a library like `jspdf` or browser `window.print()` with print-specific CSS to generate printer-ready output.
    - Ensure printed size respects ID card aspect ratio.

14.5 Admin Control for Card Design

From **Admin Dashboard**, add a section:

`/admin/settings/card-design` where admin can configure:

- Membership fee text (e.g., "ዓመታዊ 36,000 ብር")
- Current membership year text (e.g., `2018`, `2017`, etc.)
- Club contact info shown on back (address, phone, fax, PO box, email, FB).
- QR base URL (e.g., `https://fasilfans.com/verify?mid=`).
- Approver name text and signature image upload.
- Option to upload **updated background templates** if design is refreshed later.

All these values should be stored in the `Setting` table and used when rendering the card.

14.6 NFC & QR Consistency

- `membershipId` printed on card, encoded in QR, and mapped to `nfcCardUid` in DB must all refer to the same member.
- At events or gates, scanning either the **QR** or the **NFC** should let a future verification app fetch the same member record.